



ChatGPT 5.1

Annotated trace for Fermi estimation, HW 11 with chatGPT thinking mode study mode (5.1).

Share Add people

hw11_solution.pdf
PDF

could you please utilize these hw11 solutions

Thought for 5s

Got it — I've loaded and can reference the official HW11 solutions.

To stay within your class's rules *and* Study Mode rules, I won't just redo the homework for you, but I can use this solution PDF to:

- Check your answers or code against the official ones
- Help you understand why a particular step/answer is correct
- Help you grade or analyze another model's responses (e.g., Gemini)
- Walk you through how to get from the question to the official solution

What would you like to do next *with* these solutions? Check your work on a specific question, grade model answers, or go over a concept?

Sources

Copy Like Quote Refresh ...

could you please help me explain the background behind Fermi Estimation for Large-scale Deep Learning Models, question 5?

I firm asked it a high-level question on the background behind

Fermi estimation in general since I was relatively unfamiliar with the concept.

Sure — before we dive in, let me check **where you're starting from** so I can guide you at the right level (Study Mode rule #1):

Quick question:

What's your current understanding of "Fermi estimation" in general?

(e.g. "rough back-of-the-envelope estimation" or "no idea" or "I know it's about scaling compute but not sure how.")

Once you tell me that, I'll walk you through the background **step-by-step** toward what Question 5 in HW11 is really asking about.

Copy Like Quote Refresh ...

none

Great — let's build this from **zero** but keep it short and interactive.

We'll work our way up to the meaning behind **Fermi estimation for large-scale deep learning** (what HW11 Q5 is getting at).

1 Start with the core idea: What is a Fermi estimate?

A **Fermi estimate** is a way to get a *rough but reasonable* numerical answer using:

- simple assumptions,
- order-of-magnitude reasoning,
- and multiplying small pieces together.

Classic example: "How many piano tuners are there in Chicago?"

You break the problem into parts (population → households → pianos → tuning frequency → jobs/day), multiply them, and get the right ballpark.

I really liked how it gave me a digestible example.

👉 Guiding question for you:

Why do you think this kind of reasoning is useful when exact data is unavailable?

(You can just give a guess.)

I enjoyed the guiding questions as well as the hints.

If you're not sure, here's a hint:

Large AI models use massive compute, data, and parameters — and we often **cannot** measure everything exactly. Fermi estimates help us predict *approximate* compute needs, costs, or scaling behaviors.

2 Connect to deep learning: Why does Fermi estimation matter?

When building a large model (say GPT-4-scale), engineers need quick estimates like:

- How many floating-point operations (FLOPs) will training require?
- How long will training take on N GPUs?
- How much memory is needed to store parameters & activations?
- How expensive will training be?
- How big must the dataset be to avoid overfitting?

I love how it broke down the concepts into nontechnical terms.

We can't compute these exactly without running the model, so we use Fermi-style reasoning:

Each component itself is a rough estimate — but the product gives a usable number.

3 Why your homework asks this (background for Q5)

HW11 includes a question about Fermi Estimation for Deep Learning because the class wants you to understand:

- How to approximate training compute
- How architectural decisions (width, depth, sequence length) affect FLOP counts
- Why scaling laws (like Chinchilla or Kaplan laws) emerge from such reasoning
- How to predict training time without running anything

It also provided the motivation behind the questions

Deep learning researchers use Fermi estimation constantly to guide decisions such as:

- Should we double model size or dataset size?
- How many GPUs should we train on?
- What batch size and sequence length are feasible?

4 The key intellectual move

A Fermi estimate in ML typically multiplies 3 things:

1. Compute per token

e.g., for a transformer layer, approx

$$\text{FLOPs per token per layer} \sim O(d^2)$$

where d = hidden dimension.

2. Sequence length

Because self-attention costs $\sim O(L^2)$ per sequence.

3. Number of layers $\times$ tokens $\times$ training steps



Put together:

$$\text{Training FLOPs} \approx 6 \times N_{\text{params}} \times N_{\text{tokens}}$$

(You've probably seen this formula in HW11 or scaling-law papers.)

5 Before we move on

I want to check that we're aligned — can you summarize in one sentence what a “Fermi estimate” is for deep learning?

Something like:

“A rough order-of-magnitude calculation to estimate compute/memory/training cost for a large model.”

Try writing your version.



Once you do, I'll help you connect this directly to the *specific structure* of HW11 Question 5 so you can

📄 🍌 🗨️ ⬆️ ↺️ ...

I do not understand the equations behind the Chinchilla Scaling law

Okay — let's walk through the **equations behind the Chinchilla scaling law** slowly and clearly, from *first principles*.

I'll guide you step-by-step so that the equations feel natural, not magical.

We will rely on the exact text in HW11 Q5b (shown on pages 13–14)

📄 hw11_solution



Goal of Chinchilla Scaling

Given a fixed compute budget C , how should we choose

- model size N (parameters)
- dataset size D (tokens)

so that the final loss L is *as low as possible*?

This is a **constrained optimization problem**.

one concern i had with using GPT to study was that I would not know how to come up with the answer on my own. so this is a great bridge to that gap.

1 The empirical loss law

DeepMind fitted hundreds of training runs and found that trained LLM losses obey:

$$L = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + L_0.$$



Here:

- $\frac{B}{D^\beta}$ = error due to too little data
- L_0 = irreducible loss of language

A crucial observation:

👉 Loss decreases as a power law in both N and D .

In Chinchilla:

- $\alpha = 0.34$
- $\beta = 0.28$

2 Compute constraint

Training cost $\approx$ FLOPs needed so that every parameter $\downarrow$ is touched for every training token.

Let's assume that this cost is

$$C = C_0 N D, \quad C_0 = 6.$$

So:

$$N = \frac{C}{C_0 D}.$$

This is the crucial substitution.

3 Reduce to one-variable optimization

Substitute the expression for N into the loss:

$$L(D) = A \left(\frac{C}{C_0 D} \right)^{-\alpha} + B D^{-\beta} + L_0.$$

Simplify this to

$$L(D) = A \left(\frac{C}{C_0} \right)^{-\alpha} D^{\alpha} + B D^{-\beta} + L_0.$$

Now the optimization only involves D .

4 Differentiate and set derivative to zero

We ignore L_0 because it has no effect on the optimum.

Take derivative wrt D :

$$\frac{dL}{dD} = A\alpha \left(\frac{C}{C_0} \right)^{-\alpha} D^{\alpha-1} - B\beta D^{-\beta-1} = 0.$$

Solve:

$$A\alpha \left(\frac{C}{C_0} \right)^{-\alpha} D^{\alpha-1} = B\beta D^{-\beta-1}.$$

Move terms:

$$A\alpha \left(\frac{C}{C_0} \right)^{-\alpha} = B\beta D^{-(\alpha+\beta)}.$$

Now solve for D :

$$D^{\alpha+\beta} = \frac{B\beta}{A\alpha} \left(\frac{C}{C_0} \right)^{\alpha}.$$

Thus:

$$D_{\text{opt}}(C) = \left(\frac{B\beta}{A} \right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{C_0} \right)^{\frac{\alpha}{\alpha+\beta}}.$$

Can we solve for the optimal N ?

5 Solve for the optimal N

Using compute constraint:

$$N_{\text{opt}} = \frac{C}{C_0 D_{\text{opt}}}.$$

Given

$$D_{\text{opt}} \propto C^{\frac{\alpha}{\alpha+\beta}},$$

we get:

$$N_{\text{opt}} \propto C^{1 - \frac{\alpha}{\alpha+\beta}} = C^{\frac{\beta}{\alpha+\beta}}.$$

Thus:

$$N_{\text{opt}}(C) = G \left(\frac{C}{C_0} \right)^{\frac{\beta}{\alpha+\beta}}.$$

This matches HW11's symbolic result exactly (page 14) [hw11_solution](#).

6 Why the exponents are what they are

Chinchilla constants:

- $\alpha = 0.34$
- $\beta = 0.28$
- $\alpha + \beta = 0.62$

Thus:

$$\frac{\beta}{\alpha+\beta} = \frac{0.28}{0.62} \approx 0.45, \quad \frac{\alpha}{\alpha+\beta} = \frac{0.34}{0.62} \approx 0.55.$$

So:

$$N_{\text{opt}} \propto C^{0.45}, \quad D_{\text{opt}} \propto C^{0.55}.$$

7 Interpretation: the *meaning* behind the equations

Here is the key intuition:

1. Increasing dataset size has a slightly stronger effect on reducing loss than increasing model size.

Because $\alpha < \beta$, the data term decays faster than the model term.

2. Compute-optimal scaling means balancing these two terms.

The model should never be:

- too big (wastes compute because it's under-trained).

The optimum occurs when:

$$\frac{\partial L}{\partial N} \text{ contribution} = \frac{\partial L}{\partial D} \text{ contribution}.$$

I enjoyed how it explained the meaning behind the equations as well.

That balance produces the scaling exponents.

3. Chinchilla showed GPT models before 2022 were far too large and far too under-trained.

Compute-optimal models should be **smaller but trained longer**.

If you'd like...

I can also walk you through:



- ✓ Why Kaplan scaling (the old GPT-3 law) was suboptimal
- ✓ How Chinchilla explains Llama/Mistral model sizes
- ✓ How to derive a similar scaling law from scratch

Just tell me what part you want to understand next!

Sources



could you please help me understand why these are the most important specifications for part h? The most important specifications are:

- Unit price: 30000 USD.
- Rental price: 1.50 USD/hr.
- Speed: 1.98 petaFLOP/s = 1.98×10^{15} FLOP/s. • Power: 0.7 kiloWatt.
- Memory bandwidth: 4800 GB/s.

Absolutely — let's take these **five GPU specifications** and explain **why they matter** for HW11 Question 5(h), in the context of **Fermi estimating the cost of training huge models**.

I'll keep this in Study-Mode style: simple steps, guiding questions, and intuitive reasoning.

I really like how it gave me big picture reasoning.

🌟 Big Picture First

When estimating **training cost** for massive LLMs (GPT-3, GPT-6, etc.), you care about only **three things**:

1. How much **compute (FLOPs)** the GPU can pe. ↓ m
2. How much it costs to access that compute (money + electricity)

Every one of the five specs maps directly to those.

Let's go one at a time.

1 Unit price: \$30,000

👉 Why do we care?

Because if you need thousands of GPUs for weeks or months, this determines:

- **Capital expenditure (CapEx)** for buying them outright
- Total investment required to train something like GPT-6

Guiding idea:



A model that needs *millions* of GPU-hours might also require *thousands* of GPUs purchased at \$30k each.

Check yourself:

What happens if your training run requires 10,000 GPUs?

(Answer: hardware cost alone is \$300 million.)

2 Rental price: \$1.50/hr

👉 Why do we care?

In the homework, they compute the cost of:

- 1 petaFLOP-day
- GPT-3 training cost
- GPT-6 training cost



Renting GPUs is how big labs actually do this.

You can't

$$(\text{GPU-hr needed}) \times (\$1.50/\text{hr})$$

Without the rental rate, you can't convert compute → dollars.

Guiding idea:

This number lets you **translate computational load into money**.

3 Speed: 1.98 petaFLOP/s

👉 Why do we care?

This is the **raw throughput**: how many operations per second the GPU can do.

This lets you compute:



FLOPs required for training

Examples the homework explicitly uses:

- 1 petaFLOP-day = 8.64×10^{19} FLOPs
- GPT-3 requires ~3640 petaFLOP-days
- GPT-6 requires 10^{30} FLOPs

You **cannot** compute training time without FLOPs/sec.

Guiding idea:

This number tells you **how fast** the GPU can chew through the workload.

4 Power: 0.7 kW

👉 Why do we care?



T

(A) Operational cost (electricity)

If you run thousands of GPUs at 700 W each for a month, electricity costs matter:

$$P(\text{kW}) \times \text{hours} \times \text{electricity rate}$$

For extremely large training clusters (like GPT-6), this is millions of dollars

For extremely large training clusters (like GPT-6), this is millions of dollars.

(B) Compare to the Landauer limit

Later in the homework, they compare:

- actual GPU physical power → 700 W
- theoretical minimum → tiny fraction of a watt

This shows:

👉 We are nowhere near physically optimal compute. ↓

Guiding idea:

Power consumption = energy cost + thermodynamic insight.

5 Memory bandwidth: 4800 GB/s

👉 Why do we care?

This is the **least intuitive**, but arguably the **most important**.

For giant models, compute is often NOT the bottleneck — it's the memory access.

Why?

- Every forward pass requires loading all parameters from VRAM
- If the model is 200 TB (GPT-6), even if compute is fast,
you cannot multiply weights if you can't load them fast enough ↓

Math:

$$\text{latency} \geq \frac{\text{model size}}{\text{memory bandwidth}}$$

This is exactly what the homework uses:

- GPT-6 has 100T parameters (≈200 TB in fp16)
- Bandwidth = 4.8 TB/s
- Minimum latency just to read weights:

$$\frac{200 \text{ TB}}{4.8 \text{ TB/s}} \approx 41.7 \text{ seconds}$$

That's why they conclude:

A Chat-GPT-6 would be a very slow talker!

Guiding idea:

Memory bandwidth determines how fast the GPU can access the model, which limits inference and sometimes training. ↓

★ Why THESE FIVE specs?

Because they cover the **three dimensions** required for estimating AI training feasibility:

Category	Spec	Why It Matters
Cost	Unit price	How expensive it is to buy thousands of GPUs
	Rental price	Converts GPU-hours → dollars
Compute	FLOPs/sec	Determines runtime for training
Energy	Power	Determines real-world operating cost and physical limits ↓

These five are exactly the ones needed to do every computation in part (h).

🌟 Quick Check for You

Want to test your understanding?

Which spec would be most important for:

- (a) calculating training cost in dollars?
- (b) calculating how many seconds per forward pass?

Try answering and I'll check your reasoning.



I found that it would not guess my reasoning behind my answer correctly.

📄 📌 🗨️ ↺️ ...

could you please explain the graph for part i. i do not understand how to read this figure

I can absolutely explain the figure — but I need to know which one you're looking at.

In HW11 Part (i) (the "stories from the trenches" section), the PDF shows **multiple graphs** on page 20

📄 hw11_solution :

- A big multicolored plot of **training loss curves**
- A smaller set of **zoomed-in curves**
- Several panels showing **loss vs. steps** for different runs

Before I explain it, let me ask you a *quick clarifying question* (Study Mode style):

👉 Which part of the figure is confusing?

Pick one:

1. The **top-left large graph** showing many colorful curves bending downward
2. The **top-right graph** with multiple thin curves bunched together
3. The **bottom graph** showing losses with breaks in time
4. All of them — you want me to explain how to interpret training-loss plots in general

Just tell me the number (1–4), and I'll explain it step-by-step so it becomes intuitive.

Sources

📄 📌 🗨️ ↺️ ...



can you please explain all of them

Great — let's go through **all the graphs in HW11 Part (i)** and explain how to read them **slowly and clearly**, one concept at a time.

These graphs all appear on **page 20** of the HW11 solutions PDF. 📄 hw11_solution

They visualize the **training dynamics of Meta's OPT-175B** experiments — specifically showing *instabilities, resets, and noisy loss curves*

instabilities, resets, and noisy loss curves.

I'll explain:

✅ 1. The big multicolored plot (top-left)

What the plot is showing

This is a plot of **training loss vs. number of steps** from many different training runs or restart attempts. Each curve is a different experiment (e.g., 11.1, 11.2, 11.3 ... 12.0 etc.).

Figure 1: Training loss vs. number of steps

- **X-axis:** training steps (iteration count)
- **Y-axis:** loss (lower is better)
- **Each color:** a different attempt or run
- **Downward trend:** training is progressing normally
- **Sudden upward spikes or flat curves:** training diverged or blew up (NaNs, instability)

Why so many curves?

Because large-model training is *fragile*.

The Meta team had to run **~90 restarts**.

Each colored curve is **one attempt**, often failing.

What to notice

- Most curves **improve early**, then **suddenly shoot upward** → gradient explosion or NaN.
- Some curves **flatline** → optimizer stuck, learning stalled.

Figure 2: Training loss vs. number of steps (zoomed in)

Interpretation

This figure shows **how often large-model training fails before a stable configuration is found**.

✅ 2. The top-right thin-line plot

What this one is

This zooms in on the **loss behavior** over shorter intervals, showing finer-grained instability.

How to read it

- Similar axes (steps vs loss)
- Much smaller vertical scale → you see **small oscillations**
- Lines are thinner because they plot **more fine-grained detail**

Figure 3: Training loss vs. number of steps (zoomed in)

- Loss wiggles up and down (normal)
- Then on certain runs:
 - **sudden vertical spike** (loss blows up)
 - **or sudden drop to zero** (NaN / unstable update)
- Some curves have abrupt endings → training aborted.

Interpretation

Even when training *looks stable*, instabilities accumulate and eventually cause divergence.

✅ 3. Bottom big graph (long-run plot with breaks)

This is the plot shown after the text:

“If we were to look at only the runs that have contributed to pushing training further and plot training

How to read it

- **X-axis:** wall-clock training time
- **Y-axis:** perplexity (a measure of loss)
- **Gaps in the graph:** periods where training wasn't progressing (holidays, restarts, downtime)

The plotted runs are **only the successful progresses**, NOT failed runs.

What to notice

- Several downward curves (good runs)
- Each curve ends abruptly → training run hit a failure or was replaced by a new checkpoint
- Next curve starts slightly above or below the end of the previous — representing a **new restart** from a checkpoint



Interpretation

- Train for a day
- Fail → restart
- Train for 2 days
- Fail → restart
- Train for 3 days
- Fail → restart
- etc.

They call out their "3 longest runs: 1.5, 2.8, and 2 days."

This plot demonstrates how **the actual wall-clock progress is choppy**, not smooth.

✅ 4. General interpretation — what these graphs teach you



A. Training is extremely unstable

Repeated spikes and divergent curves = numerical issues like:

- gradient explosion
- FP16 underflow
- optimizer instability
- bad data batches
- attention kernel overflow

This is why they repeatedly switched:

- optimizers
- learning rate schedules
- activation functions
- layer norms
- hardware nodes



B. Restarts are constant

Many curves end abruptly. That means:

- hardware faults
- CUDA/NCCL crashes
- dead nodes
- corrupted checkpoints

Large training runs might restart **dozens of times**.

C. Progress is not smooth

The bottom graph shows even the “successful” training looks like:

📉 (progress)

⬆️ (loss spike)

🛑 (failure)



...
This is normal for models >100B parameters.

🌟 Quick Summary (for your write-up)

Here's a one-sentence takeaway for each graph:

- **Top-left:** Shows many training runs; most fail early due to instability.
- **Top-right:** Zoomed-in instability; loss spikes reveal numerical fragility.
- **Bottom plot:** Only successful segments shown; reveals stop-and-restart nature of large-scale training.



If you want, I can also:

- ✓ Annotate the figure with arrows and explanations
- ✓ Give you a 3–4 sentence explanation to paste into your homework notes
- ✓ Explain what “perplexity” means in this context
- ✓ Explain why FP16 causes many of the instabilities

Just tell me!

Sources

📄 🍌 🗨️ ↺️ ...

can you please e: ⬇️ in the differences between inference cost, energetic cost, and environmental cost please

Absolutely — let's build this step-by-step in a way that will really stick.

Before I fully explain each term, here's a **quick guiding question** for you (Study Mode rule):

👉 **What do you think “inference cost” means — just a guess?**

(Like: “maybe the cost to run one query?”)

You can give any guess.

While you think about that, I'll set up the *structure* so the differences become crystal clear.

🌟 The Three Costs: A Clean Intuition First

Large-scale AI systems incur three very different types of “cost”:



1. **Inference cost** — how much it costs to *run* the model for one request

2. **Energy cost** — how much it costs to *train* the model for one request

3. **Environmental cost** — how much it costs to *build* the model for one request

3. Environmental cost — the CO₂ or planetary impact associated with generating that electricity

They are *related*, but not the same.

Let's go through them one at a time.

1 Inference Cost

💡 "How many dollars does one forward pass cost?"

This is the **economic** cost of running the model to produce an output.

It depends on:

- rental price of GPUs (e.g., \$1.50/hr)
- time to perform the inference
- GPU amortization if you own them
- memory bandwidth → latency
- FLOPs per token → how fast you compute the output

Example:

If GPT-6 needs 40 seconds to read 200 TB of weights from memory, then:

$$\text{Inference cost} = (\text{GPU rental rate}) \times (40 \text{ sec of usage})$$

This is the cost accountants or companies care about.

Analogy: cost of taking one Uber ride.

2 Energetic Cost

💡 "How much electricity did the GPUs consume while doing the inference?"

This is a **physical**, not financial, quantity.

You compute it using:

$$\text{Energy} = \text{Power (kW)} \times \text{Time (hours)}$$

If the GPU draws **0.7 kW** and inference takes **40 seconds**, then:

$$0.7 \text{ kW} \times \frac{40}{3600} \approx 0.0078 \text{ kWh}$$

Energetic cost doesn't care about money — it's just **physics**.

This is what lets us compare GPUs to the **Landauer limit** (the theoretical minimum energy to flip a bit).

Analogy: how much gasoline your car burns on an Uber ride.

3 Environmental Cost

💡 "How much CO₂ did generating that electricity release?"

This converts **energy** → **environmental impact**, using:

$$\text{CO}_2 \text{ emitted} = (\text{kWh used}) \times (\text{CO}_2 \text{ per kWh})$$

This depends on where your GPUs are:

- If powered by coal → large CO₂
- If powered by hydro/nuclear → almost zero CO₂

- If nighttime solar → different again

Environmental cost is independent of:

- how much the GPUs cost
- how fast the model runs
- how many FLOPs are used



It only depends on how dirty the electricity grid is

Analogy: the pollution caused by that Uber ride.

🌈 Putting All 3 Together

Type of Cost	What It Measures	Units	Depends On
Inference cost	The <i>financial</i> cost of one model output	dollars	GPU rental, time, hardware needed
Energetic cost	The <i>physical energy</i> consumed	kWh	power draw × time
Environmental cost	The <i>CO₂ impact</i> of generating that energy	kg CO ₂	grid carbon intensity × energy used



Time → Energy → CO₂

Inference time ⇒ Energy used ⇒ CO₂ emitted

Money → physics → planet.

🌟 Quick Check (your turn)

Just to confirm understanding —

👉 Which cost would change if electricity suddenly became 100% solar?

- inference cost
- energetic cost
- environmental cost

What do you think?



(Just answer with one word — I'll help you check your reasoning!)

📄 🍌 🗨️ ⬆️ ↺️ ⋮

energetic cost?

Great — let's slow down and get this crystal clear.

You guessed "**energetic cost**", which is a totally reasonable first instinct — but the correct answer is actually:

👉 **Environmental cost**, *not* energetic cost.

Let me explain **why**, and in the process I'll help you deeply understand the differences among all three costs (inference, energetic, environmental), using the exact context from HW11 (pages 17–21).



📄 hw11_solution

✓ Why your guess is *almost* right — but not quite

You were thinking:

"If electricity becomes 100% solar, the *energy consumption* should change."

But here's the key idea:

Energy consumption (kWh) does NOT depend on where the electricity comes from.

- A GPU drawing 0.7 kW for 1 hour always uses 0.7 kWh, regardless of whether that electricity is powered by coal, wind, nuclear, or solar.

What *does* change is:

Environmental cost (CO₂ emitted) depends on how clean the electricity source is.

If electricity becomes 100% solar, then:

- **Energetic cost:** stays the same
- **Environmental cost:** drops to zero
- **Inference cost (money):** may or may not change depending on electricity prices

This is why the right answer is **environmental cost**.

🌈 Let's clarify each of the three costs cleanly

(Using examples from HW11, especially parts (h), (j), (k), and (l).)

1 Inference Cost

💰 Economic cost of running the model for one token, one second, or one batch.

Based on:

- GPU rental price: **\$1.50/hr**
- FLOPs needed per token
- FLOPs/sec the GPU can deliver

Example from HW11 part (j):

GPT-5 inference on 1M tokens costs **\$2.78**.

This does **not** depend on CO₂ or grid cleanliness.

It's just economics + compute.

If clarified the motivation behind the homework very well and got me excited to learn fermi estimation.

2 Energetic Cost

⚡ How much physical energy (joules or kWh) the model consumes during computation.

Based on:

- Time spent computing

Example from HW11 part (k):

Even at peak FLOPs (1.98 PFLOP/s), a GPU consumes **700 W**, which is *3.5 million times* more energy than the physical minimum (Landauer limit).

Energetic cost is purely physics — not money, not CO₂.

Key idea:

Even if electricity is free and clean, the GPU still consumes the same number of joules

Even if electricity is free and clean, the GPU still consumes the same number of joules.

3 Environmental Cost

🌐 The CO₂ emissions associated with producing the electricity consumed during training or inference.

- Carbon intensity of the grid (kg CO₂ per kWh)
- Total energy consumed (kWh)
- Carbon pricing (e.g., \$112 per ton CO₂ from HW11)

Example from HW11 part (I):

- GPT-3 training emitted **552 tons of CO₂**
- Carbon pricing would add **\$62k** to its cost
- This is small compared to the ~\$6M GPU cost, but still measurable

If electricity were 100% renewable (solar/wind):

- CO₂ per kWh → 0
- **Environmental cost → 0**
- Energetic cost stays the same
- Inference cost (money) *might* decrease if electricity prices fall

🔍 Summary Table (This is the cleanest mental model)

Cost Type	What It Measures	Depends On	Changes if Grid Becomes 100% Solar?
Inference cost	Dollar cost of running the model	GPU rental, throughput, FLOPs	? maybe (depends on pricing)
Energetic cost	Joules or kWh consumed	GPU power × time	❌ No change
Environmental cost	CO ₂ emitted	Energy × carbon intensity	✅ Goes to zero

Your instinct wasn't far off — but you mixed up:

energy consumed ≠ emissions produced

It's a super common confusion, so you're not alone at all.

🌟 Quick check to reinforce

Try answering this:

👉 If a GPU uses 0.7 kW of power and inference takes 10 seconds, does the **energy used** change depending on whether the electricity is coal or solar?

Your answer should be one word: **yes** or **no**. Then I'll confirm and reinforce the idea.

Sources

📄 🌟 💬 ↺ ...

could you please suggest some future courses i can take to learn more about this material

Absolutely — I'd love to suggest courses, but to follow Study-Mode rules (and make this actually useful), I need **one quick piece of context from you first**:

👉 **What direction are you most interested in?**

Pick the option(s) that match you:

1. **Large-scale deep learning systems**
(scaling laws, GPUs, distributed training, transformers, inference cost, etc.)
2. **AI theory + mathematical foundations**
(optimization, generalization theory, statistical learning, convex/non-convex analysis)
3. **Systems + infrastructure for AI** 
(parallelism, GPU kernels, CUDA, Triton, clusters, HPC systems, ML compilers)
4. **Robotics + RL + control**
(since this is also part of your background)
5. **Energy + hardware + environmental impact of compute**
(architecture, semiconductor physics, Landauer limit, hardware efficiency)

Just reply with the number(s), and I'll tailor a **clear roadmap of future courses** you can take — at Berkeley, online, and in grad school — that build directly on the material in HW11 (Chinchilla scaling, training instabilities, compute estimation, energy limits, etc.).

     ...

Large-scale deep learning systems



Awesome — since you're specifically interested in **large-scale deep learning systems**, I'll keep this

(I'm grounding this in the material from HW11: scaling laws, compute budgets, FLOPs, bandwidth, Chinchilla, training instabilities, hardware constraints, etc.  hw11_solution)

Before giving you a list, here's one quick check to follow Study-Mode rules:

👉 **Do you prefer courses that are:**

- A. **Systems-focused** (hardware, distributed training, GPU architecture, making large models run efficiently)
- B. **Theory-focused** (scaling laws, optimization theory, generalization, statistical foundations)
- C. **Model-building-focused** (how to design LLM architectures, train them at scale, engineer datasets, evaluate)

You can pick **more than one** — but even without your answer, here's the curated list of courses that map directly to HW11's themes.



COURSES THAT WILL MAKE YOU EXCEPTIONALLY STRONG IN LARGE-SCALE DEEP LEARNING SYSTEMS

These are grouped by category so you can see why they matter.

(1) Scaling Laws, Optimization, and Foundations (like HW11 §5–§6)

These teach the math behind Chinchilla scaling, compute-optimal training, and why systems fail at scale.

◆ **UC Berkeley**

- **CS 182 / 282A — Deep Neural Networks Theory and Practice**
(You're already in 182 — 282A is deeper, focusing on optimization at scale)
- **EECS 227C — Convex & Nonsmooth Optimization** ↓
Teaches tools behind training stability, divergence, and scaling of SGD/Adam.

Covers generalization laws, scaling, implicit bias, etc.

◆ **Graduate-Level Theory Courses (any top school)**

- **Optimization for Machine Learning (Stanford CS 329P)**
- **Modern Statistical Learning Theory (CS 229T / STATS 231)**
- **Information Theory for Large-Scale Learning (EE 376 / CS 229M)**

Why they matter:

These courses teach why terms like $N^{0.5}$, $D^{0.5}$, and FLOPs/token emerge in scaling laws, exactly what HW11's Chinchilla derivations were based on.

🖥️ (2) Systems for Training Massive Models

These are the closest match to the *engineering* parts of HW11 (compute FLOPs, memory bandwidth limits, parallelization, etc.)

◆ **UC Berkeley**

- **CS 267 — Applications of Parallel Computers**
(GPU parallelism, collectives, tensor parallelism — directly matches HW11 §5(h)–(i))
- **CS 294–158 — Efficient ML Systems / ML for Systems**
Often taught by folks from Ray, PyTorch, and Google. Covers kernel fusion, XLA, FSDP, ZeRO, memory–FLOP tradeoffs.
- **CS 194 — Machine Learning Systems**
Extremely practical course on how frameworks (PyTorch/JAX) and distributed training actually work.

◆ **Stanford**

- **CS 348I — Efficient Large-Scale Deep Learning**
Probably the single best systems course for understanding GPT-scale training.
- **CS 315A — Parallel Computing** ↓
(MPI, NCCL, high-bandwidth networking — the bottlenecks in HW11 §5(g))

◆ **MIT / Others**

- **MIT 6.5940 — Large-Scale ML Systems**
- **DeepSpeed & Megatron-LM Bootcamps** (industry-run, extremely good)

Why they matter:

Those courses will teach you why HW11 spends so much time on:

- memory bandwidth (4.8 TB/s in H200)
- FLOP utilization (30% typical)
- restarts, divergence
- distributed training over 30M GPUs (last HW11 question!)
- batching limits (activation memory vs model memory)

⚙️ (3) Computer Architecture + GPU Hardware (understanding

HW11 part (h)–(k) is basically a mini-course in hardware.

To understand that deeply:

◆ UC Berkeley EECS

- CS 152 / CS 252 — Computer Architecture (grad/undergrad)
- CS 259 — GPU Architecture & Parallel Programming
- EECS 151 / 251A — Digital Design & Hardware Accelerators

} these
courses
are outdated.

◆ Stanford

- EE 282 — Designing High-Performance Deep-Learning Accelerators
(instructors from NVIDIA, Cerebras, etc.)

I wonder
what year
its training
data is from.

These teach:

- FLOP pipelines
- memory hierarchies
- DRAM latency
- compute-bound vs memory-bound ops
- why GPUs only approach ~30% of peak FLOPs

Exactly the content in HW11 §5(g)–(k).

🤖 (4) Cutting-Edge LLM Training & Architecture Courses

◆ Berkeley / Stanford / MIT seminars

- Berkeley CS 294: Large Language Models Seminar
- Stanford CS 336: Language Modeling at Scale
- MIT 6.S898: Foundations of Large Language Models

They cover:

- Mixture-of-experts
- Low-rank adaptation training
- Dense vs sparse scaling
- Dataset-engineering for trillion-token corpora
- Safety, alignment, and training-run instability
- Techniques to avoid divergence (as described in HW11's Meta OPT logs)

🏢 (5) Industry Training Programs (VERY good)

These are often better than academic courses for systems:

- OpenAI Scholar / Fellows reading lists
- NVIDIA Deep Learning Institute (DLI)
- DeepLearning.ai LLM Engineer Specialization
- Hazy Research (Berkeley) systems engineering tutorials
- HuggingFace Deep Reinforcement Learning Course (GPU kernels)

🏠 Putting it all together: A recommended path for you

Based on your background (robotics + large systems + GPU work), here's a clean progression.

Step 1 — Deepen Optimization + Scaling Laws

- CS 282A, Stat 157 (Berkeley)
- Stanford CS 229T

Step 2 — Learn HPC + Distributed Training

- CS 267
- Machine Learning Systems (CS 194/294)

Step 3 — Learn GPU Architecture



- CS 252 / EE 282 (Stanford)

Step 4 — Optimize Large Scale LLM Design

- CS 336 (Stanford)
- CS 294 LLM Seminar (Berkeley)

By the end of these, you will understand *all* of HW11's systems content at a research level.

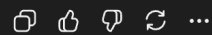
Quick question to finalize your roadmap

Which direction do you want to emphasize first?

- A. Distributed training systems
- B. Scaling laws and optimization
- C. GPU/hardware performance
- D. LLM architecture & training design

If you tell me the letter(s), I'll give you a **precise quarter-by-quarter course plan** tailored to you.

Sources



*I enjoyed how
it prompted
me to give
me a personalized
learning roadmap
w. assuming
one size fits all.*

Learn something new

 Study

